Reviewer Pack — Minimal Order of Local Units in Quandle Theory and Communica…

Entry 83b5cdb25351 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-07 18:51:59 UTC

Minimal Order of Local Units in Quandle Theory and Communication Complexity Rank Variance Bound

Entry ID: `83b5cdb25351` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-07 18:51:59 UTC

1. Conjecture

Field A (mathematical branch): Quandle Theory **Field B** (complexity object): Communication Complexity (Matrix Rank)

Statement:

> For every Boolean function f with communication complexity rank r_f, the minimal order of local units in the quandle associated with f is linearly correlated with r_f, such that min_order(quandle(f)) = Θ(r_f).

Rationale (proposer's reasoning):

> Quandles are algebraic structures that capture certain properties of combinatorial configurations and their symmetries. The conjecture posits that the order of local units in a quandle associated with a Boolean function can provide insights into the communication complexity rank of the function, offering a new perspective on understanding computational tasks through algebraic structures.

Taxonomy category: `ADJUNCT_RELATIONS` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `a04e44e541c47e8c`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if at least 80% of the quandles Q_f have a minimal order of local units within a factor of 3 from their corresponding communication complexity rank r_f, across all seeds. The conjecture is falsified if any seed produces a quandle with a minimal order of local units that exceeds its communication complexity rank by more than 10.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	1.00	SAFE	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=124, elapsed=240.1s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_boolean_function(n):
        return [random.choice([0, 1]) for _ in range(2**n)]

    def communication_complexity_rank(f):
        n = int(math.log2(len(f)))
        rank = 0
        for i in range(2**n):
            for j in range(i+1, 2**n):
                if f[i] != f[j]:
                    rank += 1
        return rank

    def quandle_order(f):
        n = int(math.log2(len(f)))
        order = 0
        for i in range(2**n):
            for j in range(i+1, 2**n):
                if f[i] != f[j]:
                    order += 1
        return order

    def min_order(quandle):
        return len(quandle)

    instances_tested = 0
    n_max = 0
    total_min_order = 0
```

```
    total_rank = 0

    for _ in range(30):
        n = random.choice([5, 10, 15, 20, 30, 40])
        f = generate_boolean_function(n)
        rank = communication_complexity_rank(f)
        quandle = [f[i] ^ f[j] for i in range(2**n) for j in range(i+1, 2**n)]
        min_order_val = min_order(quandle)

        instances_tested += 1
        n_max = max(n_max, n)
        total_min_order += min_order_val
        total_rank += rank

    mean_min_order = total_min_order / instances_tested
    mean_rank = total_rank / instances_tested
    support_fraction = (abs(mean_min_order - mean_rank) <= 3 * math.sqrt((mean_min_order**2 + mean

    conjecture_holds = support_fraction >= 0.8
    counterexample = "" if conjecture_holds else "mapping_undefined"

    return {
        "metric_name": "min_order_vs_rank",
        "metric_value": mean_min_order,
        "instances_tested": instances_tested,
        "n_max": n_max,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    if len(sys.argv) > 1:
        seeds = [int(s) for s in sys.argv[1:]]
    else:
        primes = [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71, 73,
        seeds = random.sample(primes, 30)

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
    std_metric_value = math.sqrt(sum((r["metric_value"] - mean_metric_value)**2 for r in results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(s for s, r in zip(seeds, results) if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample='mapping_undefined' first_failing_seed={first_fai
    else:
        print("RESULT: INCONCLUSIVE mapping_undefined")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
TIMEOUT after 240s
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test timed out before producing data, which means it was unable to complete the required computations to verify the conjecture. | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 12

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14140
2	propose	ollama_remote	glm4:latest	0	0	13270
3	propose	ollama_remote	glm4:latest	0	0	17497
4	propose	ollama_remote	glm4:latest	0	0	11909
5	preregistration	ollama_remote	glm4:latest	0	0	9573
6	novelty	ollama_remote	glm4:latest	0	0	8761
7	novelty	ollama_remote	glm4:latest	0	0	8442
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13645
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	44468
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10220
11	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12440
12	judge	ollama_remote	glm4:latest	0	0	74930

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 239295 ms total latency. Provider mix: {'ollama_remote': 12}

(full prompt+response transcripts available in `research/audit/83b5cdb25351.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/83b5cdb25351.jsonl` for verification of provenance

4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/83b5cdb25351.tar.gz` (if generated)